# Team sf-pet-care

## 1. Team Members:

Ahmad Naggayev
Zhidan Niu
Maurice Onyonyi
Jonah Robinson
Thomas Sasser

## 2. URLs:

**Git Repo:**
https://github.com/Morioh/sf-pet-care
**GCP Project:**
https://console.cloud.google.com/welcome/new?authuser=1&project=sf-pet-care

## 3. Introduction

The purpose of this project is to see if different San Francisco neighborhoods have access to pet care services. These services include veterinary clinics, grooming shops, and pet stores. We compare service availability across the official neighborhoods in San Francisco. We use multiple data sets, including Geo Location, business and census data. This information is useful for pet owners who want to know which neighborhoods have better access to various pet oriented services.

## 4. Data Sources and Acquisition

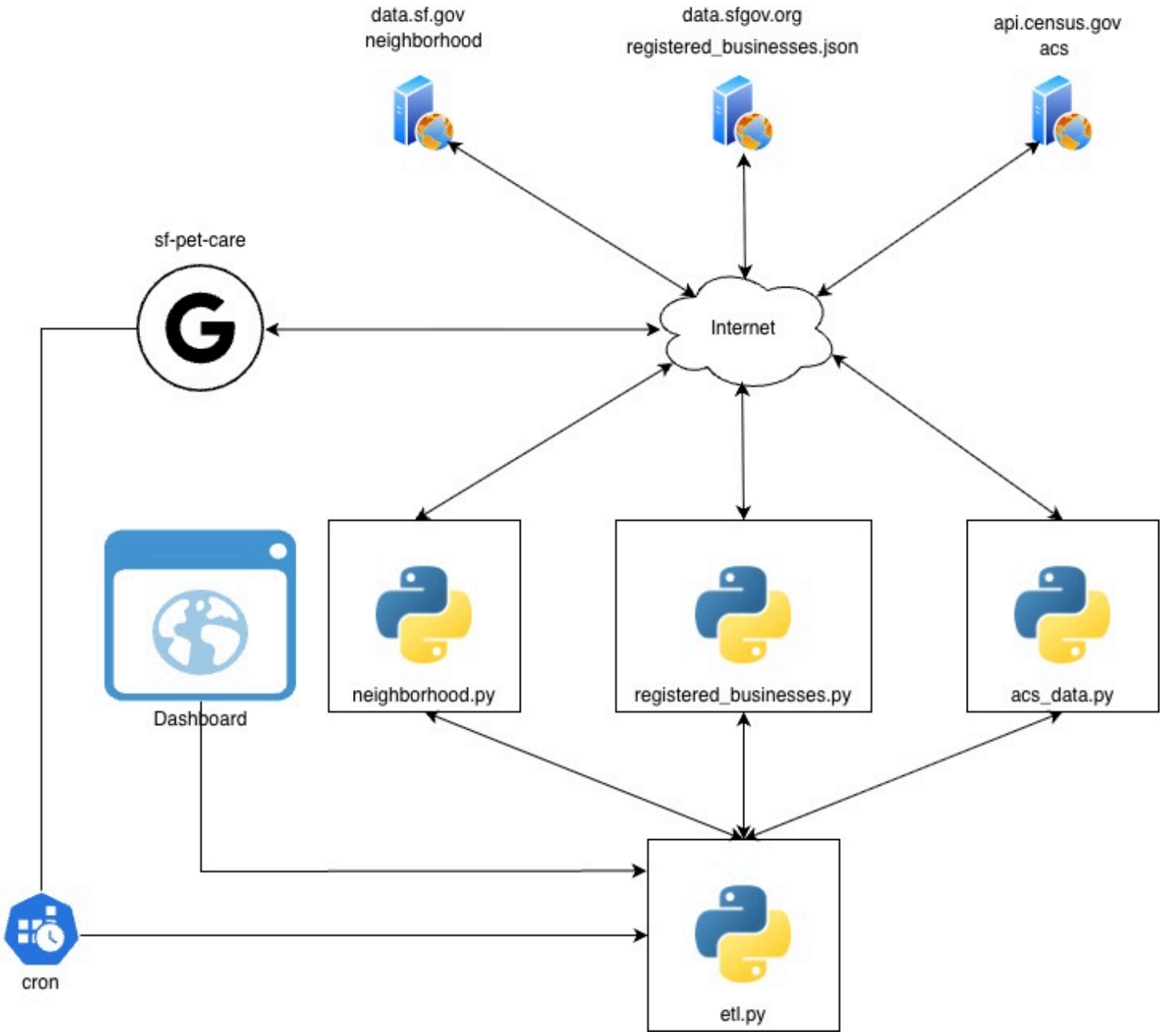
**Source:** Registered Business Locations — San Francisco
**URL:** **https://data.sfgov.org/resource/g8m3-pdis.json**
**Method:** API
**Contains:** Registered businesses in San Francisco including: business name, address, business information, dates, and geographic location
**Update Frequency:** Daily
**Access Requirements:** Public; no login or API key required

**Source:** American Community Survey 2024 5 Year Estimates
**URL:** **https://api.census.gov/data/2024/acs/acs5**
**Method:** API
**Contains:** Population and demographic information for small geographic areas
**Update Frequency:** Daily
**Access Requirements:** Requires API key

**Source:** San Francisco Analysis — Neighborhoods
**URL:** https://data.sf.gov/api/v3/views/j2bu-swwd/query.geojson?accessType=DOWNLOAD
**Method:** File
**Contains:** Names and geographic boundaries of San Francisco neighborhoods

**Update Frequency:** Daily
**Access Requirements:** Public, no key or login

## 5. System Architecture Diagram




The center of the system is the etl.py script. It is a FastAPI application with endpoints to call each data gathering script. The data gathering scripts are neighborhood.py, registered_businesses.py, and acs_data.py. These scripts are responsible for fetching data from the source site, uploading and downloading the transformed data to the GCP cloud bucket. These calls are accessed from the endpoints in the FastAPI application. Scheduled jobs in GCP have several options: one endpoint for each script and one aggregated endpoint that runs all scripts.

The neighborhood.py script has two library function interfaces: upload_neighborhood() and download_neighborhood(). The former function downloads data from any URL via Pandas gpd.read_file(URL). Data transformation consists of dropping extraneous columns, renaming and sorting them. The upload function gets the GCP client, bucket and file name before converting the GeoPandas geo-dataframe to JSON. It is then uploaded to the bucket as a file.

The acs_data.py script is more complicated. The fetch_acs_data() function downloads the Census data via API. It uses an API key issued by the Census Bureau. It uses requests to GET the data and returns JSON. The clean_acs_data() function performs transformation on the Pandas data frames; it renames some columns, edits invalid data, and adds a new column 'geoid' that is a combination of state, county and tract data. A separate helper function makes a request to get latitude and longitude data for each tract, we then join these two data frames. It uploads data to the GCP bucket as a JSON file. Download is the same in reverse.

Finally, registered_businesses.py pulls data from data.sfgov.org and looks for active pet care businesses. It uses an API token for authentication to increase the rate limit. The fetch_registered_businesses_data_from_api() function downloads and parses the data. The upload and download functions add data to the GCP bucket, or download it, respectively.

The streamlit app calls the endpoints hosted by FastAPI. We have a variety of GET and POST URLs that it uses to answer user questions.

**6. Technical Implementation**

The download schedule only exists as a requirement for the project. Frequent updates are not necessary for the data, but do serve as a good example. The Census data is updated every few years. Only the business data needs to be on a more frequent schedule, but once a week should be more than sufficient. We set it to daily just for the example project. In actual production, monthly updates would be reasonable.

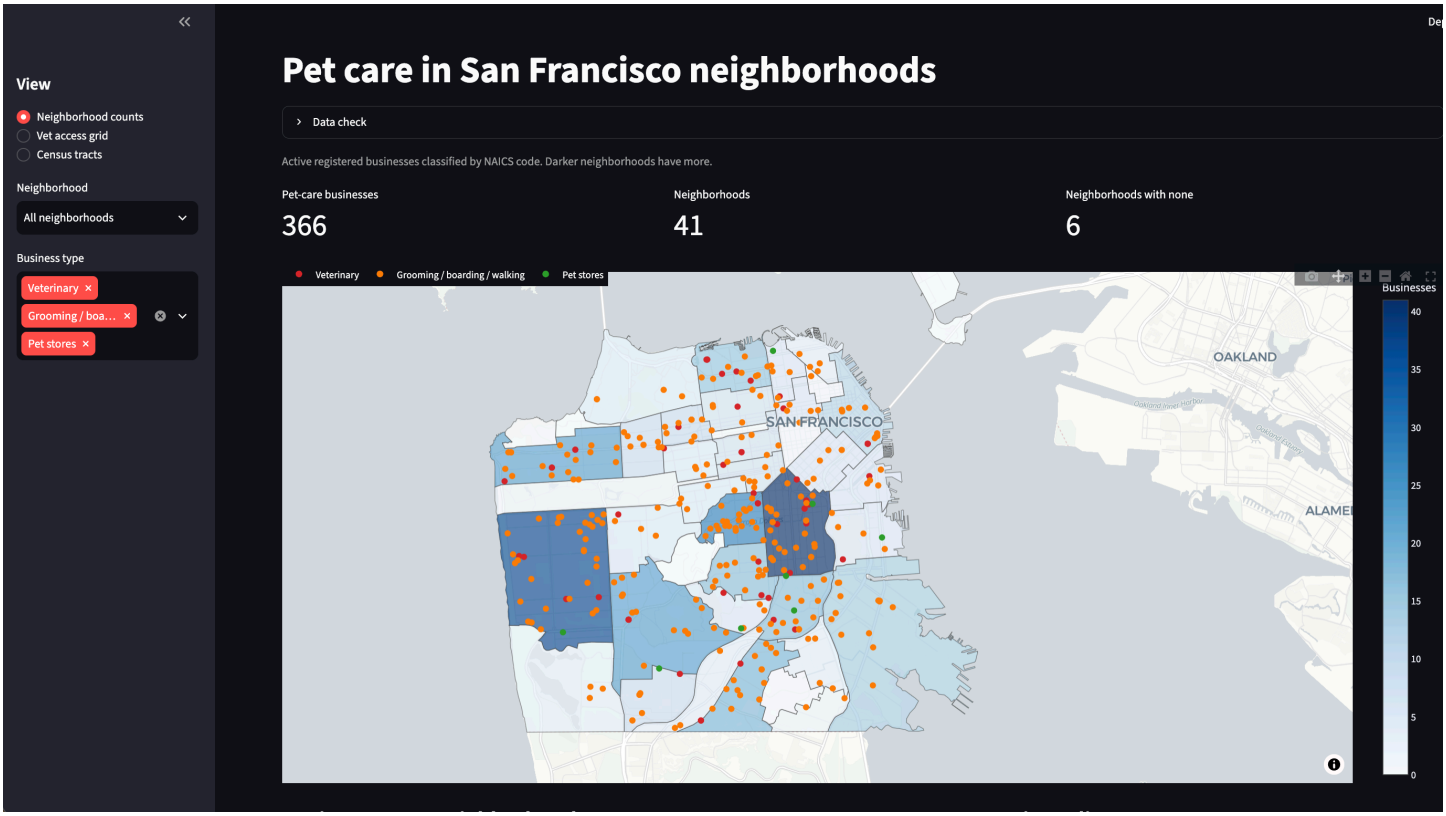
All our data is processed in some way. These are light edits designed to make the data more accessible for the dashboard. The data was mostly clean to begin with. The sources have done a good job with data design and access.

The primary design decision was to aggregate all the fetch scripts under etl.py. Ideally, the project would only need one cronjob since the script calls all three fetch scripts. However, for the purposes of the project, the etl.py script offers an endpoint for each one that cron can run. The dashboard will make all calls for the data via the FastAPI endpoints rather than going to the GCP bucket directly.

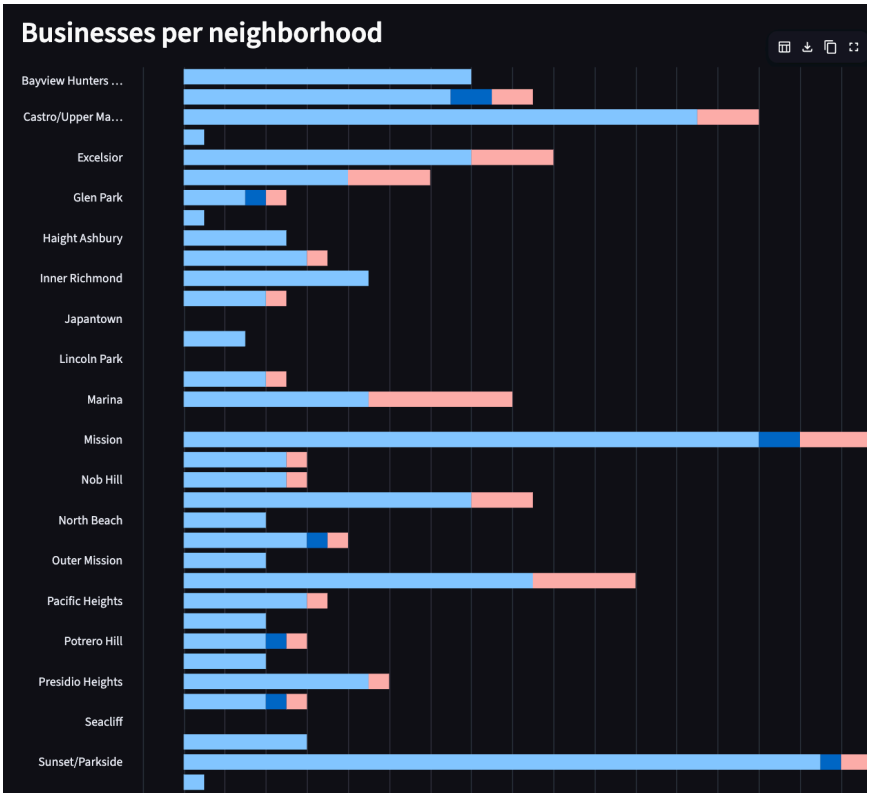
**7. Results**

The data can actually be visualized through streamlit. Three main views were made, which collectively make use of the acs tract data, the neighborhood data and the registered business data. The three views are respectively called: Neighborhood counts, vet access grid and Census tracts, and can be toggled under view.

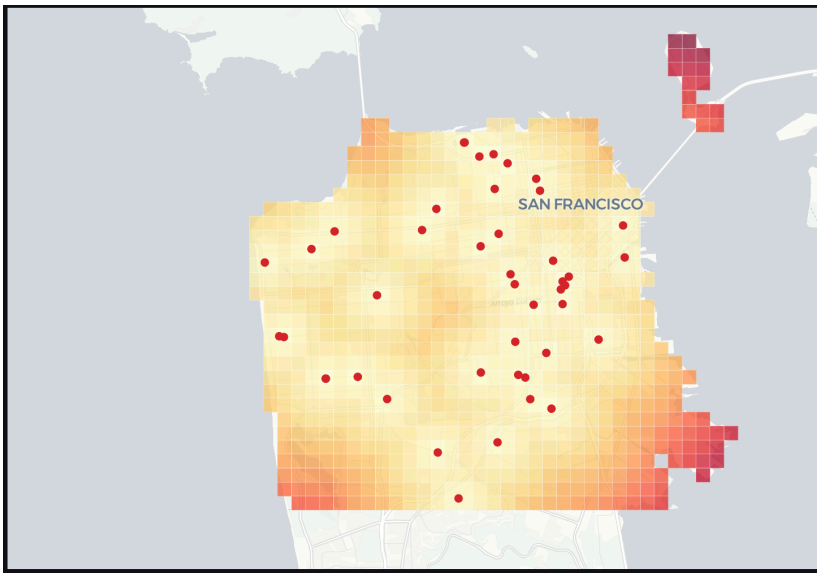
The first view showcases a map of San Francisco with dots overlaid on top which represent different pet businesses: segmented through color by whether it's veterinary, grooming or pet store. Business

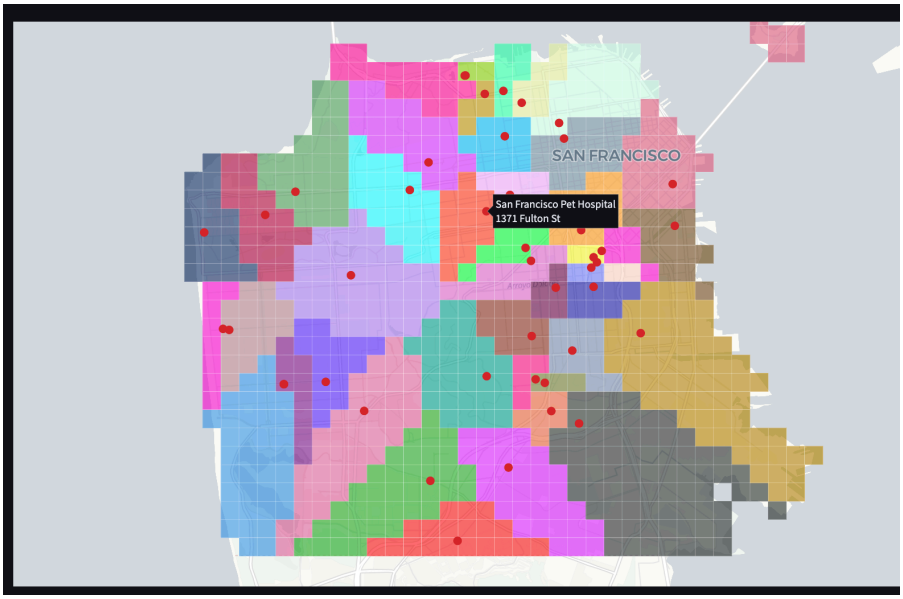



types can be selected or deselected. The neighborhood boundaries are also drawn, showing a darker hue in neighborhoods with more pet care. Some insights about the data is that most pet businesses are grooming or care shops. There are then less vets, while pet stores are the least frequent.

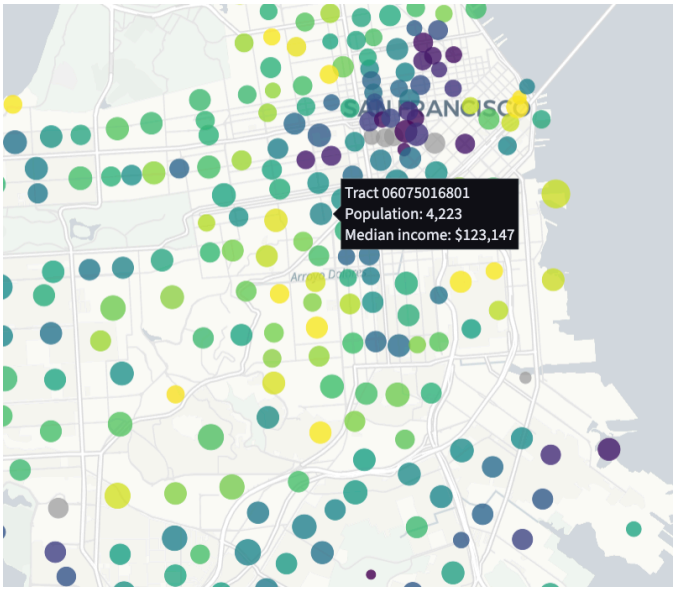


The second view is called Vet Access Grid and essentially divides San Francisco into many different small grids, and for each grid calculates the distance to the closest vet. Darker reds are used to indicate further distance. The findings show that in general, due to an abundance of vet clinics across the city most areas are relatively close to vets, with the furthest neighborhoods being about 3 miles away, and found in treasure island, and the southeastern and south western corners of the city. but most areas being less than one mile. There's also an alternate view that shows groups together grids which share the same closet veterinary clinic.

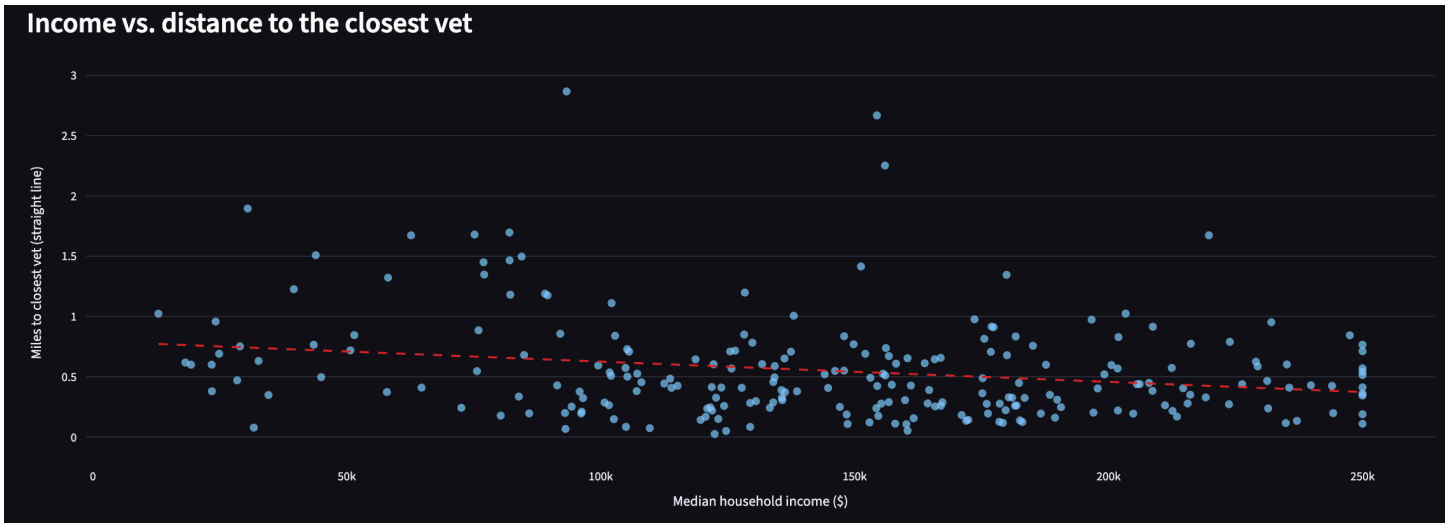


The third view is called Census Tracts, it overlays dots over the map which each represent different Census Tracts. The shape of the tracts aren't displayed but their latitude and longitude is used to inform where the dots are placed. These are much more granular than neighborhoods as each represents only a few thousand residents. For each census tract you can learn the population and median income.




Finally, we wanted to plot and see if there was a correlation between the income of a tract, and the distance to the closest veterinary clinic. A negative correlation does exist: lower income tracts tended to be further away from vets, but the correlation is weak.




Example API Response to the endpoint: /neighborhoods


```
{"type": "FeatureCollection", "features": [{"id": "0", "type": "Feature", "properties": {"neighborhood":
"Bayview Hunters Point"}, "geometry": {"type": "MultiPolygon", "coordinates": [[[[-122.38157774241415,
37.75307043091241], [-122.38156949251606, 37.753060959298274], [-122.38159239626694, 37.75309424492931],
```

[-122.38155614326205, 37.753045901366754], [-122.38155137472305, 37.75304127677009], [-122.38154650687193, 37.753036719547374], [-122.3815415385967, 37.75303223061754], [-122.38153647334677, 37.75302781172802], [-122.38153131112232, 37.75302346287867], [-122.38152605423795, 37.753019185835115], [-122.38152070389688, 37.75301498328154], [-122.38151526236827, 37.75301085518192], [-122.38150973196667, 37.75300680330162], [-122.38150411158058, 37.75300282855954], [-122.38149840577098, 37.7529989317842], [-122.38149261460651, 37.75299511567804], [-122.38148674033339, 37.75299137930393], [-122.38148078528907, 37.75298772532827], [-122.38147475063121, 37.7529841546

## 8. Challenges, Lessons Learned, and Next Steps

The primary problem was understanding requirements. We operated under incorrect requirements for a significant portion of the project. While it did cause some difficulties, it did drive a better design by moving us to the etl.py script and its endpoints.

The team created a full sprint using Slack's Project Tracker. This worked very well. It was good practice for how actual teams work. The engineers needed to be prompted to edit and close their user stories, as usual. However, they are now accustomed to working from stories.
Additionally, the team found that paired programming worked very well. Much of the code needed to be rewritten for clarity and understanding. It was a great instructive tool for the less technical team members. Those engineers stated it helped them quite a bit.

Another lesson is that the team was far more effective in person. Instant communication allowed for immediate changes and/or answers to questions.

Some of the GCP interactions were difficult. The engineers needed to spend some time with the scheduler. We also had trouble with the GCloud configuration.

There is no future work planned for this project. It's a basic app that has some fun information.

## 9. Ethics

None of this document, along with the README.md, was written with AI. Everything was hand-written. Any portion from a previous assignment that had been written with AI was changed.

The streamlit front end was written with AI, as allowed in the requirements. This is noted in the file. The prompt is in /docs as Claude.md.